Linear Programming Examples 2

Lecture 4

ETF2480&ETF5248 - S2, 2026
Jessica Leung, PhD, CAP-X, SFHEA

Today's Goal



- Feasibility problem
- Behind the scene: Simplex Algorithm

Feasibility Problem

We talked about feasibility problem in Lecture 2. Let's look at an example of one.

Leon Burnit advertising agency

Leon Burnit adveritsing agency is trying to determine a TV advertising schedule for Priceler Auto Company. Priceler has three goals:

- its ads should be seen by at least 40 million high impact manager (HIM)
- its ads should be seen by at least 60 million leaders in private sector (LIP)
- its ads should be seen by at least 35 million health informatics worker (HIW)

Leon Burnit can purchase two types of ads: those shown during football games and those shown during soap operas where, at most, $600,000 can be spent on ads.
The advertising costs and potential audiences (in millions of viewers) of a one-minute ad of each type are shown in the table

ad	HIM	LIP	HIW	cost, $
football	7	10	5	100,000
soap opera	3	5	4	60,000

Leon Burnit must determine the feasibility of this plan.

Leon Burnit advertising agency: formulation

Let

- x_1 be the number of minutes of ads shown during football games
- x_2 be the number of minutes of ads shown during soap operas

Then we can formulate Leon Burnit's problem as:

$$
\begin{aligned}
\text{minimize} \quad & 0 \\
\text{subject to} \quad & 7x_1 + 3x_2 \geq 40 \\
& 10x_1 + 5x_2 \geq 60 \\
& 5x_1 + 4x_2 \geq 35 \\
& 100x_1 + 60x_2 \leq 600 \\
& x_1, x_2 \geq 0
\end{aligned}
$$

What can you say about the feasible set?

Simplex Algorithm

Ever wondered how R is solving the LPs behind the scene?

Simplex algorithm

- invented in 1947 (George Dantzig)
- usually developed for LPs in standard form ('primal' simplex method)
- can also be developed for inequality form LP ('dual' simplex method)
- in fact, simplex is a family of algorithms

Step 1. Convert LP to standard form.

Step 2. Obtain a starting bfs (if possible) from the standard form.

Step 3. Determine whether the current bfs is optimal.
if optimal, **quit**
else, **continue**

Step 4. Determine which nonbasic variable should enter (become basic) and which basic variable should exit (become nonbasic) to find a bfs with a better objective function value.

Step 5. Pivot, i.e., find a new bfs and return to Step 3.

Simplex algorithm

key idea: move from a bfs to an adjacent bfs with better objective value

questions:

- how do we find a starting bfs?
  if all constraints are $\leq$, use slack variables as basic
  otherwise, use big-M or two-phase method (see later)
- when does the algorithm terminate, i.e, when are we optimal?
  for a max. problem: a bfs is optimal if each nonbasic variable has a nonnegative coefficient in row 0
- how do we find an entering variable?
  choose nonbasic var. with the smallest negative coefficient in row 0
- how do we find an exiting variable?
  choose the winner of the ratio test
- how do we pivot?

Example 1: Dakota furniture company

The Dakota Furniture company manufactures desk, tables, and chairs. The manufacture of each type of furniture requires lumber and two types of skilled labor: finishing and carpentry. The amount of each resource needed to make each type of furniture is given in the table below.

Currently, 48 board feet of lumber, 20 finishing hours, 8 carpentry hours are available. A desk sells for $60, a table for $30, and a chair for $20. Dakota believes that demand for desks and chairs is unlimited, but at most 5 tables can be sold. Because the available resources have already been purchased, Dakota wants to maximise total revenue.

resource	**desk**	**table**	**chair**
lumber (board feet)	8	6	1
finishing (hours)	4	2	1.5
carpentry (hours)	2	1.5	0.5

Example 1: problem formulation

$$
\begin{array}{lrrrrl}
\text{maximize} & 60x_1 + & 30x_2 + & 20x_3 & & \\
\text{subject to} & 8x_1 + & 6x_2 + & x_3 & \leq & 48 \\
& 4x_1 + & 2x_2 + & 1.5x_3 & \leq & 20 \\
& 2x_1 + & 1.5x_2 + & 0.5x_3 & \leq & 8 \\
& & x_2 & & \leq & 5 \\
& & & x_i & \geq & 0
\end{array}
$$

equivalent to:

$$
\begin{array}{lrrrrrrrrrl}
\text{maximize} & 60x_1 + & 30x_2 + & 20x_3 & & & & & & & \\
\text{subject to} & 8x_1 + & 6x_2 + & x_3 + & s_1 & & & & & = & 48 \\
& 4x_1 + & 2x_2 + & 1.5x_3 & & + & s_2 & & & = & 20 \\
& 2x_1 + & 1.5x_2 + & 0.5x_3 & & & & + & s_3 & = & 8 \\
& & x_2 & & & & & & + & s_4 = & 5 \\
& x_1, & x_2, & x_3, & s_1, & s_2, & s_3, & & s_4 \geq & & 0
\end{array}
$$

Example 1: simplex tableau

$$
\begin{array}{lrrrrrrrrrrrr}
\text{maximize} & z = & 60x_1 + & 30x_2 + & 20x_3 & & & & & & & & \\
\text{subject to} & & 8x_1 + & 6x_2 + & x_3 + & s_1 & & & & & = & 48 \\
& & 4x_1 + & 2x_2 + & 1.5x_3 & + & s_2 & & & & = & 20 \\
& & 2x_1 + & 1.5x_2 + & 0.5x_3 & & + & s_3 & & & = & 8 \\
& & & x_2 & & & & + & s_4 & = & 5 \\
& & x_1, & x_2, & x_3, & s_1, & s_2, & s_3, & s_4 & \geq & 0
\end{array}
$$

resulting simplex tableau:

Table: simplex tableau (initial)

	z	x_1	x_2	x_3	s_1	s_2	s_3	s_4	**rhs**	**BV**
row 0	1	-60	-30	-20	0	0	0	0	0	z = 0
row 1	0	8	6	1	1	0	0	0	48	s_1 = 48
row 2	0	4	2	1.5	0	1	0	0	20	s_2 = 20
row 3	0	2	1.5	0.5	0	0	1	0	8	s_3 = 8
row 4	0	0	1	0	0	0	0	1	5	s_4 = 5

Example 1: step 3, 4

Table: simplex tableau (initial)

	z	x_1	x_2	x_3	s_1	s_2	s_3	s_4	rhs	BV
row 0	1	-60	-30	-20	0	0	0	0	0	$z = 0$
row 1	0	8	6	1	1	0	0	0	48	$s_1 = 48$
row 2	0	4	2	1.5	0	1	0	0	20	$s_2 = 20$
row 3	0	2	1.5	0.5	0	0	1	0	8	$s_3 = 8$
row 4	0	0	1	0	0	0	0	1	5	$s_4 = 5$

- Is the current bfs optimal?
  no, three nonbasic variables have negative coefficients in row 0
- Choose entering variable
  x_1 has the smallest negative coefficient in row 0
- Choose exiting variable: perform ratio test to determine how large we can make the entering variable:
  as x_1 increases, the values of the current BVs change to satisfy equations
  need to make sure current BVs remain nonnegative, for example:

$$s_1 \geq 0 \implies x_1 \leq 48/8 \iff x_1 \leq 6$$
$$s_2 \geq 0 \implies x_1 \leq 20/4 \iff x_1 \leq 5$$

Example 1: step 3, 4

Table: simplex tableau (initial)

	z	x_1	x_2	x_3	s_1	s_2	s_3	s_4	**rhs**	**BV**
row 0	1	-60	-30	-20	0	0	0	0	0	$z = 0$
row 1	0	8	6	1	1	0	0	0	48	$s_1 = 48$
row 2	0	4	2	1.5	0	1	0	0	20	$s_2 = 20$
row 3	0	2	1.5	0.5	0	0	1	0	8	$s_3 = 8$
row 4	0	0	1	0	0	0	0	1	5	$s_4 = 5$

- Is the current bfs optimal?
  no, three nonbasic variables have negative coefficients in row 0
- Choose entering variable
  x_1 has the smallest negative coefficient in row 0
- Perform ratio test:
  row 1: $x_1 \leq 48/8 = 6$
  row 2: $x_1 \leq 20/4 = 5$
  row 3: $x_1 \leq 8/2 = 4$
- Row 3 wins the ratio test, so s_3 will exit (become nonbasic)

Example 1: step 5

Pivot in the entering variable:

- Make the entering variable basic in a row that wins the ratio test.
- Use elementary row operations to turn the x_1 coefficient in row 3 to 1.
  for example, divide row 3 by 2.
- Use elementary row operations to turn the rest of x_1 coefficients to 0.
  for row 1: multiply the new row 3 by 8 and subtract from row 1.
  for row 2: multiply the new row 3 by 4 and subtract from row 2.

Table: simplex tableau (iteration 1)

	z	x_1	x_2	x_3	s_1	s_2	s_3	s_4	**rhs**	**BV**
row 0	1	0	15	-5	0	0	30	0	240	$z = 240$
row 1	0	0	0	-1	1	0	-4	0	16	$s_1 = 16$
row 2	0	0	-1	0.5	0	1	-2	0	4	$s_2 = 4$
row 3	0	1	0.75	0.25	0	0	0.5	0	4	$x_1 = 4$
row 4	0	0	1	0	0	0	0	1	5	$s_4 = 5$

Example 1: step 5

Table: simplex tableau (iteration 2)

	z	x_1	x_2	x_3	s_1	s_2	s_3	s_4	**rhs**	**BV**
row 0	1	0	5	0	0	10	10	0	280	$z = 280$
row 1	0	0	-2	0	1	2	-8	0	24	$s_1 = 24$
row 2	0	0	-2	1	0	2	-4	0	8	$x_3 = 8$
row 3	0	1	1.25	0	0	-0.5	1.5	0	2	$x_1 = 2$
row 4	0	0	1	0	0	0	0	1	5	$s_4 = 5$

is the current bfs optimal?

- yes, all nonbasic variables have nonnegative coefficients in row 0

remarks:

- after each pivot, each constraint must have a nonnegative rhs
- the coefficient of a decision variable in row 0 is often referred to as the variable's **reduced cost**
- binding constraints have slack variables equal to 0

Complexity

worst-case:

- for most pivoting rules, there exist examples where the number of iterations grows exponentially with n and m
- it is an open question whether there exists a pivoting rule for which the number of iterations is bounded by a polynomial of n and m

in practice: very efficient (# of iterations typically grows linearly with m, n)